

Task Board Project Report

Full-Stack Django & React Enhanced Application Documentation

1. Project Overview

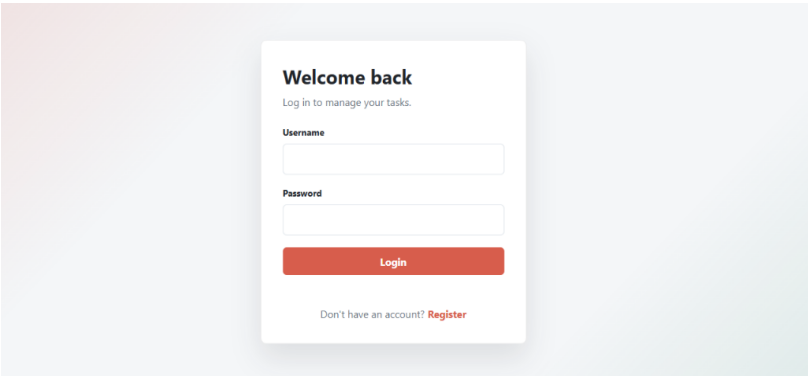
The Task Board application is a full-stack task management web app built with a Django backend and a React single-page frontend. Recent enhancements introduced Task Priority levels (High, Medium, Low), Due Date tracking with status alerts, Dashboard analytics cards, dynamic completion progress bar, advanced filter tabs, instant completion toggling, and Dark Mode theme switching.

2. Technology Stack & Features

Component	Technology Used	Description
Backend	Python 3, Django 6	Database models, CSRF handling, and REST JSON APIs
Frontend	React 19, Vite	Component views, route state, interactive UI elements
Database	SQLite 3	Stores User and Task records (with Priority & Due Date)
Styling	Custom CSS & Variables	Responsive layout, Dark Mode theme, Priority badges

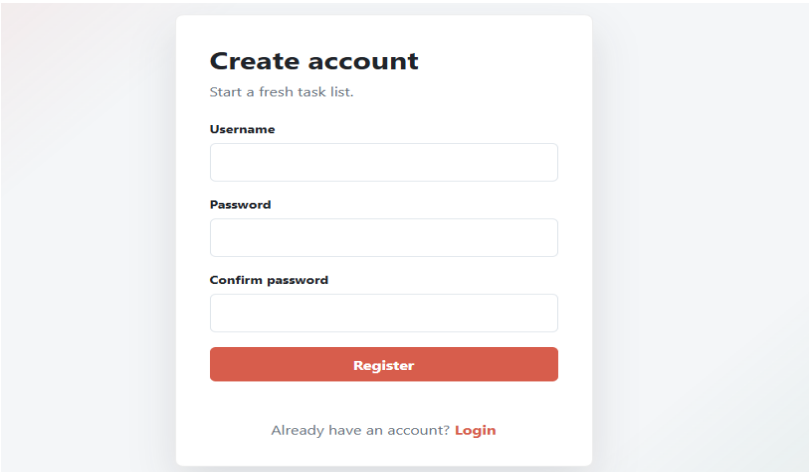
3. Application Screenshots & Explanations

Figure 1: Login Page



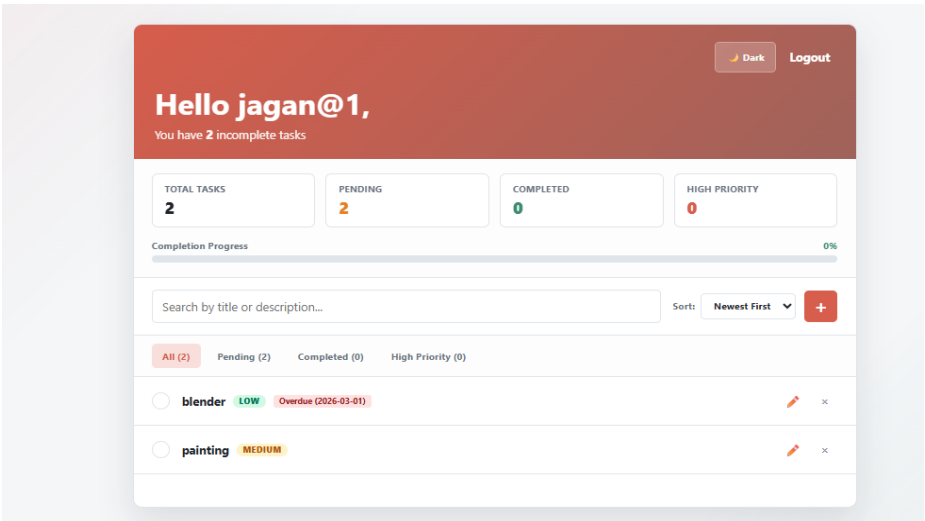
Login Page Explanation: Provides authentication interface. Users enter Username and Password to log into their account. Submitting sends a POST request to /api/login. On valid credentials, the app navigates to the user's task dashboard.

Figure 2: Registration Page



Registration Page Explanation: New users create an account with Username, Password, and Confirm Password fields. Submitting sends a POST request to /api/register. The backend verifies password matching and username uniqueness.

Figure 3: Enhanced User Dashboard



Enhanced Dashboard Explanation: Shows topbar greeting with Dark Mode toggle and Logout. Features 4 analytics cards (Total Tasks, Pending, Completed, High Priority), a dynamic Completion Progress Bar (0%), search filter input, sort dropdown, filter tabs (All, Pending, Completed, High Priority), quick checkbox toggles, color-coded priority badges (e.g. LOW, MEDIUM), and due date alerts (e.g. Overdue (2026-03-01)).

Figure 4: Enhanced Create Task Form

The screenshot shows a 'Create task' form with a white background and rounded corners. At the top left is a red 'Back' button. Below it is the title 'Create task' in bold, followed by the subtitle 'Add something useful to your list.' The form contains a 'Title' input field, a 'Description' text area, a 'Priority' dropdown menu with 'Medium' selected, a 'Due Date' input field with a placeholder 'dd-yyy-yyyy', a 'Mark as Completed' checkbox, and a red 'Create Task' button at the bottom.

Create Task Form Explanation: Allows users to input Title, Description, select a Priority level (High, Medium, Low), pick an optional Due Date, and check completion status before submitting to `/api/tasks/<user_id>/create`.

Figure 5: Enhanced Update Task Form

The screenshot shows an 'Update task' form with a white background and rounded corners. At the top left is a red 'Back' button. Below it is the title 'Update task' in bold, followed by the subtitle 'Adjust the details, priority, and status.' The form contains a 'Title' input field pre-filled with 'blender', a 'Description' text area, a 'Priority' dropdown menu with 'Low' selected, a 'Due Date' input field pre-filled with '01-Mar-2026', a 'Mark as Completed' checkbox, and a red 'Save Changes' button at the bottom. The form is displayed on a desktop screen with a Windows taskbar visible at the bottom.

Update Task Form Explanation: Pre-loads existing task details (e.g. title 'blender', Priority 'Low', Due Date '01-Mar-2026'). Users can adjust title, description, priority, due date, and status, then save changes via `/api/tasks/<task_id>/update`.

Figure 6: Delete Task Confirmation

The screenshot shows a 'Delete task' confirmation modal with a white background and rounded corners. At the top left is a red 'Go Back' button. Below it is the title 'Delete task?' in bold, followed by the subtitle 'This will remove the task from your list.' The modal contains a red box with the task name 'painting' and two buttons at the bottom: 'Cancel' and 'Delete'.

Delete Task Page Explanation: Confirmation modal displaying the task name with Cancel and Delete buttons. Clicking Delete sends a POST request to `/api/tasks/<task_id>/delete`.

4. Automated Testing Verification

All backend API endpoints and SPA routes were verified using Django's test framework (python manage.py test):

Test Function	Status	Details
test_api_register	PASSED	Verifies user creation via API
test_api_login	PASSED	Verifies user login authentication
test_api_create_task	PASSED	Verifies task creation with Priority and Due Date
test_api_update_task	PASSED	Verifies updating task title/status/priority
test_api_toggle_task	PASSED	Verifies instant task completion toggle
test_api_delete_task	PASSED	Verifies task deletion from database
test_react_app_route	PASSED	Verifies SPA template serving